

GITOPS-FIRST PLATFORM ENGINEERING

AKS Cluster Management

KRO + ASO • Kyverno • Argo
Workflows



The Challenge

Traditional cluster management at scale



Individual pipelines triggered manually for each cluster



Inconsistent configs, no single source of truth



No automated health checks or certification

CURRENT SCALE

1,500+

Clusters to consolidate

TARGET STATE

<20

Consolidated clusters

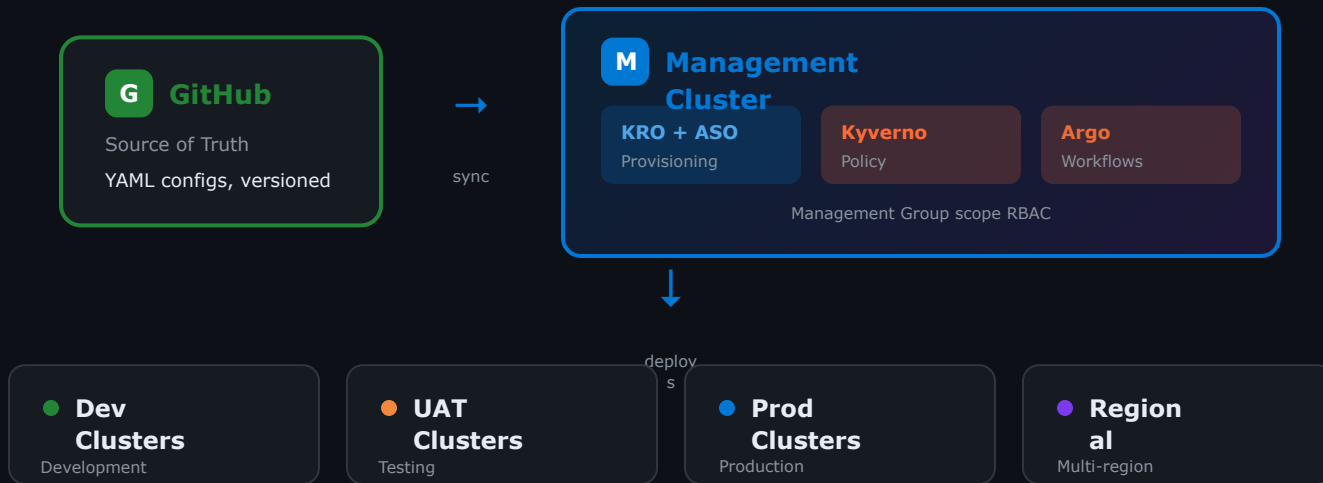
GLOBAL USERS

1000s

Developers worldwide

GitOps-First Architecture

Management cluster controlling worker clusters via declarative config



No manual pipelines — All changes flow from Git commits through automated sync

KRO + ASO

Cluster Resource Unit with Azure Service Operator

What KRO + ASO Provisions

AKS clusters with full configuration, node pools, networking, identity, Azure resources (DNS, Storage, KeyVault), and Kubernetes version management.

KRO Composition

ResourceGraphDefinitions compose multiple ASO resources into a single CRD. Request one "Cluster" and get AKS + networking + RBAC + monitoring.

Management Cluster Privileges

Workload Identity with Management Group scope RBAC to provision and configure all worker clusters.

EXAMPLE: CLUSTER DEFINITION

```
apiVersion: platform.xxx.com/v1
kind: ManagedCluster
metadata:
  name: prod-west europe-01
spec:
  environment: production
  region: west europe
  kubernetesVersion: "1.30"
  features:
    istio: true
    keda: true
```

One YAML = Full AKS cluster with all dependencies

Updates via simple YAML:

K8s versions

Node pools

Security

Features



Policy Engine — Admission control, mutation, generation

Validate

Block non-compliant resources: enforce security contexts, require labels, validate image registries

Mutate

Auto-modify resources: image registry swap, inject sidecars, add default security contexts

Generate

Auto-create resources: NetworkPolicies on new namespaces, RBAC bindings, default quotas

KEY USE CASES

EPA Policies

Auto-generate network policies for namespace isolation

Image Swap

Redirect to approved internal registries

Example: Generate NetworkPolicy

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: generate-default-netpol
spec:
  rules:
    - name: default-deny-egress
      match:
        resources:
          kinds: [Namespace]
      generate:
        kind: NetworkPolicy
        synchronize: true
```

Security by default — Every namespace gets isolation policies automatically

A Argo Workflows

Certification Workflows — Automated cluster health verification

Certification Workflow

Automated verification that clusters meet operational standards after provisioning or updates.

Checks: API server connectivity, node health, core addon status, network connectivity, security policy compliance

CronJob Trigger

Scheduled runs validate all clusters continuously

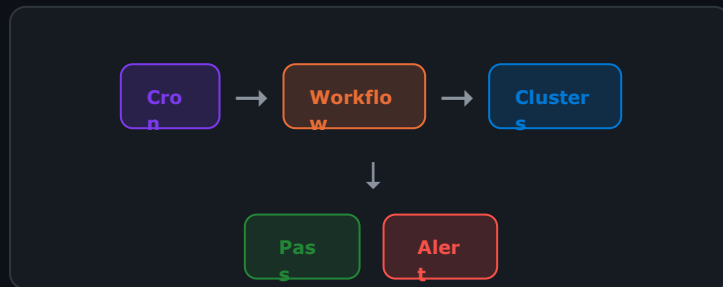
Alert Integration

Failed certifications trigger monitoring alerts

Future: Namespace Onboarding

Can extend to handle namespace provisioning with approval gates

WORKFLOW EXECUTION



Continuous assurance — Know immediately when a cluster drifts from expected state

GitOps Flow

From Git commit to running infrastructure — no manual steps



What Can Be Updated via Git Push

Kubernetes
versions

Node pool
configs

ASTL versions

Security
policies

Network
policies

New API
features

Addon
configs

Environment
groups

Releases per Environment

Tag releases (v1.2.3) for prod, track HEAD for dev

Group Targeting

Roll out changes to cluster groups by region or tier

Full Audit Trail

Complete Git history of every infrastructure change

Key Benefits

Why this approach matters for enterprise infrastructure

No Manual Pipelines

Commit to Git and infrastructure updates automatically. No triggers, no manual steps.

Simple YAML Config

Update K8s versions, security config, node pools with straightforward YAML changes.

Highly Scalable

Manage hundreds of clusters from one plane. Same effort for 5 clusters or 500.

Security by Default

Kyverno policies automatically enforce security standards across all clusters.

Continuous Certification

Argo workflows verify cluster health continuously. Immediate drift detection.

Full Audit Trail

Git history provides complete audit trail. Every change tracked and versioned.

Easy to manage, highly scalable, and robust — a GitOps-first approach that eliminates manual overhead while maintaining enterprise-grade control.